EduGear Backend — Deployment Checklist

Railway · two environments · A4 Technologies · prepared 6 October 2026

> **The shape:** one Railway *project* with two *environments* — `production` (deploys from `main`) and `staging` (deploys from `develop`). Each gets its **own** Postgres and Redis. Staging is what your frontend engineer builds against; it carries demo data and Swagger, and it never shares a database with production.

1 Before you touch Railway

- [ ] Confirm CI is green on `main` — it now builds the production image *and boots it* until `/api/health` answers.
- [ ] Create a `develop` branch from `main` if one does not exist: `git switch -c develop && git push -u origin develop`
- [ ] Generate two different JWT secrets — never reuse one across environments: `openssl rand -base64 48` (run it twice, keep them apart)
- [ ] Decide the two front-end origins (e.g. `https://app.edugear.ng` and `https://staging.edugear.ng`).
- [ ] Create a Sentry project and copy its DSN. One DSN serves both; the environments are told apart by `SENTRY_ENVIRONMENT`.
- [ ] Create **two** Cloudflare R2 buckets: `edugear-prod` and `edugear-staging`. *Staging runs NODE_ENV=production, so without R2 every upload is refused and your FE engineer cannot test photos.*

2 Databases

Environment	Recommended	Why
production	Neon Postgres	Point-in-time restore and branching. You hold fee records and receipts; "restore to 10 minutes ago" is the feature that matters when something goes wrong.
staging	Railway Postgres	Cheap and disposable. It is meant to be reset.
both	Railway Redis (one per environment)	Required in production — the app refuses to boot without it, because rate limiting and the permission cache would silently become per-instance.

- [ ] Provision the production database first and keep its credentials out of the staging environment entirely.
- [ ] Add Redis to **each** environment separately — do not point both at one instance.
- [ ] Note both connection strings: the pooled one for `DATABASE_URL`, the direct/unpooled one for `DIRECT_URL`. *Migrations need the unpooled connection.*
- [ ] Turn on automated backups for production and note the retention window.

> **Never share a database between environments.** Staging gets reset and seeded with demo schools whose password is published in the README. One shared connection string is all it takes for that to become production data.

3 Create the Railway service

- [ ] New project → deploy from this GitHub repository. Railway reads `railway.json` and builds from the `Dockerfile`.

- [ ] Name the service `edugear-backend`. *If you name it something else, set the GitHub variable RAILWAY_SERVICE to match.*
- [ ] Create the second environment: **Settings → Environments → New**, call it `staging`.
- [ ] Point `production` at the `main` branch and `staging` at `develop`.
- [ ] Attach Postgres and Redis to each environment.
- [ ] Confirm these came from `railway.json` and did not need setting by hand: `preDeployCommand: npx prisma migrate deploy`, `healthcheckPath: /api/health`, restart `ON_FAILURE` ×3.

4 Variables

Set these on each environment separately. `PORT` is injected by Railway — do not set it.

Variable	production	staging
`NODE_ENV`	`production`	`production` *(deliberate — see note)*
`DATABASE_URL`	Neon pooled URL	`${{Postgres.DATABASE_URL}}`
`DIRECT_URL`	Neon *unpooled* URL	same as DATABASE_URL
`REDIS_URL`	`${{Redis.REDIS_URL}}`	`${{Redis.REDIS_URL}}`
`JWT_ACCESS_SECRET`	secret #1	secret #2 — **different**
`APP_URL`	production API URL	staging API URL
`FRONTEND_URL`	`https://app.edugear.ng`	`https://staging.edugear.ng`
`CORS_ORIGINS`	the live front end only	staging front end + `http://localhost:5173`
`SWAGGER_ENABLED`	`false`	`true` *(the FE engineer's API reference)*
`EMAIL_PROVIDER`	`resend`	`console` *(no real email to parents from staging)*
`RESEND_API_KEY`	live key	—
`EMAIL_FROM`	your verified sender	—
`PAYSTACK_SECRET_KEY`	`sk_live_…`	`sk_test_…`
`PAYSTACK_CALLBACK_URL`	live front-end return page	staging return page
`R2_ACCOUNT_ID`	same Cloudflare account for both	
`R2_BUCKET`	`edugear-prod`	`edugear-staging`
`R2_ACCESS_KEY_ID` `R2_SECRET_ACCESS_KEY`	a separate token per bucket, each scoped to that bucket only	
`SENTRY_DSN`	the same DSN in both	
`SENTRY_ENVIRONMENT`	`production`	`staging`
`APP_VERSION`	`${{RAILWAY_GIT_COMMIT_SHA}}`	
`STORAGE_QUOTA_MB`	`2048` (or your plan)	`512`

> **Why staging runs `NODE_ENV=production`.** It makes staging behave like production: Redis is required, logs are JSON, stack traces never reach a client, and uploads need real object storage. A staging environment that behaves differently from production tells you nothing. Your FE engineer still gets a `requestId` in every error body — quote it and the full detail is in Sentry. `SENTRY_ENVIRONMENT` is what keeps staging noise out of your production alerts.

5 Wire up deployment from CI

- [ ] Service → Settings → Deploy → **Pre-deploy Command**: `npx prisma migrate deploy`. *Do not assume railway.json provides it — check the field is populated, on both environments.*
- [ ] Service → Settings → Deploy → **Healthcheck Path**: `/api/health`. *Without it Railway reports a deploy successful when the container merely started, not when the app works.*
- [ ] Service → Settings → Deploy → **Restart Policy**: On Failure, 3 retries.
- [ ] On `production`, enable **Wait for CI** on the GitHub trigger. *Without it Railway deploys on push whether the tests passed or not.*
- [ ] Leave the trigger **on** for `staging`, so the FE engineer always has the latest `develop`.

6 First deploy

- [ ] Deploy `staging` first. Always.
- [ ] Watch the logs for `Permission catalogue synced` and `Nest application successfully started`.
- [ ] Check health: `curl https://<staging>/api/health` → expect `database` and `cache` both `up`.
- [ ] Open `https://<staging>/api/docs` and confirm Swagger loads.
- [ ] Only then deploy `production`, and repeat the health check.
- [ ] Confirm `https://<production>/api/docs` returns **404**. *Swagger must be off in production.*

7 Seed staging for the frontend engineer

```
# Railway → staging → Postgres service → Variables → DATABASE_PUBLIC_URL
DATABASE_URL="<staging DATABASE_PUBLIC_URL>" APP_ENV=staging npm run db:seed
```

> **Not `railway run npm run db:seed`.** That runs the command *locally* with the environment's variables, so `DATABASE_URL` arrives as `postgres.railway.internal` — resolvable only from inside Railway. It cannot run inside the container either: the seed goes through `ts-node`, a devDependency the production image prunes. So it runs from a developer machine, over the public proxy.

- [ ] Run the seed **against staging only**. It creates two demo schools — two tenants, so cross-tenant mistakes show up immediately.
- [ ] Give the FE engineer: the staging base URL, `/api/docs`, and the demo logins. Password: `Demo9Access2026`
- [ ] Confirm their `localhost:5173` is in the staging `CORS_ORIGINS`.
- [ ] Show them the reset procedure for when they corrupt the data (below).

> **The seed refuses to run against production** — it checks `NODE_ENV` and the shape of `DATABASE_URL`. That guard exists because `railway run` uses whichever environment the CLI is linked to, and a wrong link looks exactly like a right one. Run `railway status` before anything destructive.

Resetting staging

```
DB="<staging DATABASE_PUBLIC_URL>"
DATABASE_URL="$DB" DIRECT_URL="$DB" npx prisma migrate reset --force
DATABASE_URL="$DB" APP_ENV=staging npm run db:seed
```

8 Create the platform owner (production)

```
railway environment production
railway run npm run platform:grant -- ops@a4technologies.ng OWNER --create
```

- [ ] Save the printed password somewhere safe, then sign in at `POST /api/platform/auth/login` and change it.
- [ ] Repeat for staging with a **different** email and password.
- [ ] Confirm the account belongs to no school — the command refuses otherwise, and that separation is what stops a compromised school login reaching the platform console.

9 Paystack

- [ ] Paystack dashboard → **Test mode** → set the webhook URL to `https://<staging>/api/webhooks/paystack`
- [ ] Paystack dashboard → **Live mode** → set the webhook URL to `https://<production>/api/webhooks/paystack`
- [ ] Make a test card payment on staging and confirm the invoice settles and a receipt number is issued.
- [ ] Check the audit log for `payment.online_settled`.

10 Prove it works — production smoke test

- [ ] `/api/health` returns 200 with database and cache `up`.
- [ ] `/api/docs` returns 404.
- [ ] Register a throwaway school, admit a student, raise an invoice, record and verify a payment — confirm the receipt number is issued and the balance reaches zero.
- [ ] From a second school's token, request the first school's student → expect **404**.
- [ ] Post a webhook with a forged signature → expect **401**.
- [ ] Request `/api/platform/schools` with a school token → expect **404**.
- [ ] Trigger an error and confirm it appears in Sentry tagged `production`, with no email address, request body or IP in it.
- [ ] Suspend the throwaway school from the platform console, confirm its login stops immediately, then cancel it.

11 Day two

- [ ] Set a Sentry alert for any new issue in `production`, routed somewhere a person reads.
- [ ] Confirm the first automated database backup has actually run.
- [ ] Rehearse a restore into a scratch database — a backup you have never restored is a hope.
- [ ] Record who holds the Railway token, the Paystack live key and the platform owner login.
- [ ] Diarise the JWT secret rotation. *Changing it signs every user out; do it deliberately, not in an incident.*
